

Using Sanity.io with Vanilla JavaScript for a Static Portfolio Site

Overview of Sanity.io for a Portfolio Website

Sanity.io is a headless CMS (Content Management System) that allows you to manage content (like portfolio projects, blog posts, etc.) in an online content database (the “Content Lake”) and retrieve it via APIs ¹. This decouples your content from your website’s code – you can update text, images, and project details in Sanity’s studio interface, and your site can fetch and display those updates without needing a code change. For a developer portfolio, this means you can define content types for things like **Projects**, **Blog Posts**, or **Profile** info in Sanity, add content through Sanity’s admin UI, and then use vanilla JavaScript on a static site to query and render that content on your pages. The result is a flexible, dynamic site: you use Sanity for content management and a static front-end for speed and simplicity.

How it works: You’ll set up a Sanity project (which includes a content editing **Studio** and the backend Content Lake). In the Studio, you define **schemas** for your content types (for example, a schema for “Project” with fields like title, description, image, etc.). Editors (or you) input content via these schemas and publish it. The content is stored in Sanity’s cloud database and made available through real-time APIs. On the website side, you can retrieve this content using Sanity’s APIs – either by using Sanity’s JavaScript client library or by making direct HTTP requests (using `fetch`) to Sanity’s Content Lake API. We’ll walk through setting everything up from scratch, defining schemas for a portfolio, and fetching the data using vanilla JS (with both the Sanity client and direct fetch calls).

Setting Up a Sanity.io Project from Scratch

1. Install Sanity CLI and create a project: Sanity requires Node.js (version 20 or above) and npm installed on your system ². The simplest way to start is using the **Sanity CLI’s create command**. Open a terminal in the folder for your portfolio site and run:

```
npm create sanity@latest
```

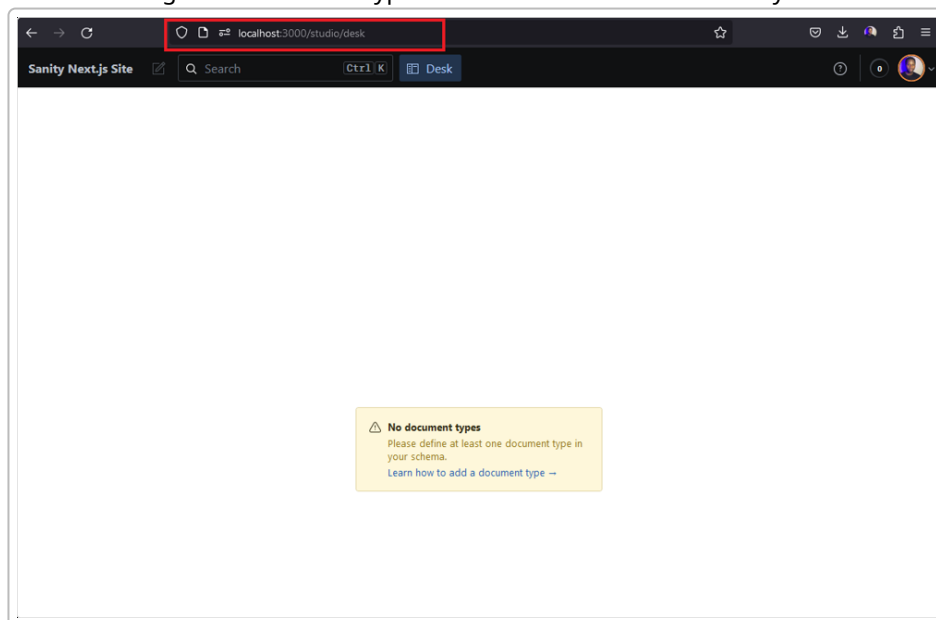
This one command will bootstrap a new Sanity Studio project (you don’t need to globally install the CLI separately in most cases). The CLI will prompt you with a series of questions to configure your project ³ ⁴. It will ask you to log in to your Sanity account (it will open a browser for authentication if needed) and then ask for a project name, dataset name, and template. For a fresh portfolio, you can choose a **clean project template** (the CLI may have an option like “clean” or “empty” with no pre-built schema) ⁴. For example, you might select the default dataset name “production” and choose the “clean” starter so you can define your own content types from scratch. (Using `--template clean` in the create command would auto-select a blank studio with no sample schemas ⁴.) You can also provide a project name during setup –

e.g., “Portfolio Site” – or via a flag like `--create-project "Portfolio Site"` to automate that ⁵, but it's fine to just answer the prompts interactively.

During this setup, the CLI will create a new Sanity project (on Sanity's cloud) and a local Studio application in the directory. It typically creates a new folder (often named after your project or a default like `my-sanity-project`). Once the CLI is done, navigate into the project folder (the CLI output will tell you the directory created). Then run the development server for Sanity Studio:

```
cd my-sanity-project # (replace with your actual folder name)
npm run dev
```

This launches the Sanity Studio at a localhost address (usually `http://localhost:3333` or `http://localhost:3000/studio` depending on configuration). You should see the Sanity Studio web app open, which is the admin interface where you define schemas and input content. At first launch, the studio may be blank or show a message “No document types” because we haven't added any content schema yet



. (Sanity Studio is a single-page application that can be embedded in frameworks like Next.js or run standalone. For simplicity, we consider it running on its own local dev server.)

2. Define your content schemas: The next step is to define the structure of the content you want to manage – in Sanity, this is done by creating **schema** files. Sanity will have a `/schemas` directory (or similar) in the project folder. You'll create JavaScript (or TypeScript) files there for each content type. Each schema describes a document type and its fields. Sanity comes with many built-in field types (string, text, image, number, datetime, boolean, reference, etc.) that you can use in your schemas ⁶. For a portfolio site, you'll likely need schemas for things like **Project**, **BlogPost**, and maybe a **Profile** or **About** document. Let's create a simple schema for a Project as an example.

```
// schemas/project.js (or project.ts if using TS)
export default {
  name: "project",          // Internal name of the content type
  title: "Project",         // Human-readable title in the studio
  type: "document",         // This schema represents a document (content item)
  fields: [
    {
      name: "title",
      title: "Title",
      type: "string"         // short text field
    },
    {
      name: "description",
      title: "Description",
      type: "text"           // longer text field (multi-line)
    },
    {
      name: "image",
      title: "Project Image",
      type: "image",         // image upload field
      options: { hotspot: true } // enable hotspot for cropping in the studio
      // You can also add fields to images, e.g., an "alt" text:
      // fields: [{ name: "alt", title: "Alt Text", type: "string" }]
    },
    {
      name: "projectUrl",
      title: "Project URL",
      type: "url"            // link to the live project or repo
    },
    {
      name: "technologies",
      title: "Technologies Used",
      type: "array",
      of: [{ type: "string" }] // an array of strings for tech tags
    }
  ]
  // ...you can add more fields like date, roles, etc. as needed
}
}
```

This schema defines a **Project** document with five fields: a string title, a text description, an image, a URL, and an array of strings (for listing technologies or tools used). In Sanity Studio, each field will correspond to an input widget (text boxes, image uploader, etc.) for content editors ⁷ ⁶. The `name` is the key used in queries and data, `title` is the label shown in the Studio UI, and `type` specifies the kind of data. Every document schema has `type: "document"` and usually a required `name` (must be unique across schemas) and a `title` ⁸.

You would create similar schema files for other content types, for example a **BlogPost** schema with fields like `title`, `slug` (for URL path), `publishedDate`, `body` (which could be rich text), etc., or a **Profile** schema for your personal info (as in some portfolio sites). For instance, a Profile schema might include fields for your name, bio, profile picture, social media links, etc. (The FreeCodeCamp guide on Sanity + Next.js provides a detailed example of a profile schema with many fields like `fullName`, `headline`, `profileImage`, `socialLinks`, etc. ⁹ ¹⁰ – you can customize based on what you want to manage via CMS.)

3. Register the schemas: Sanity requires you to register these schema definitions in the Studio configuration so it knows about them. Typically, there's an `schemaTypes` array or an `index.js` in the `schemas` folder that imports all your schema objects and exports them. For example, you might have a `schemas/index.js` like:

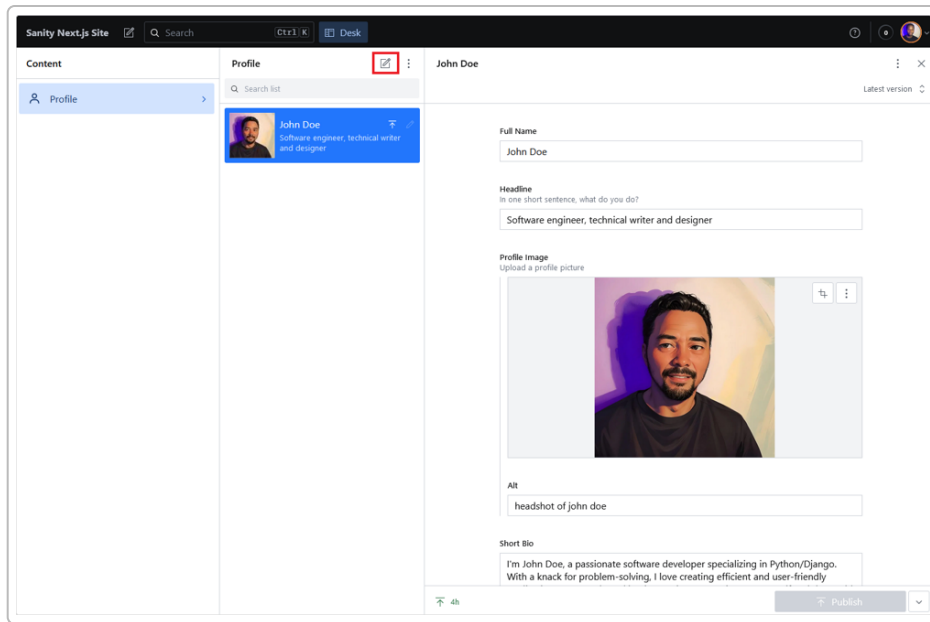
```
import project from './project'
import blogPost from './blogPost'
import profile from './profile'

export const schemaTypes = [project, blogPost, profile]
```

If you used the Sanity create CLI with a clean template, it likely set up a basic schema import/export for you (check the generated files). Ensure your new schema is included – otherwise, the Studio won't show it. Once that's done, restart the Studio dev server (if it's not hot-reloading automatically) with `npm run dev` again. Now the Studio should recognize your content types.

Populating Content via the Sanity Studio

With schemas in place, the Sanity Studio UI will display your new content types in the sidebar. You can now create and manage content. For example, you could add a few Project documents for your portfolio. Open the Studio in your browser (e.g., `http://localhost:3333`), and you should see your content types (like "Project") listed. Clicking "Create new Project" will bring up a form with the fields you defined.



Sanity Studio – Example content entry form. In the Studio's editing interface, you can input content for your portfolio. For instance, the screenshot above shows a "Profile" document being edited with fields like Full Name, Headline, Profile Image, etc. After filling in the fields, clicking Publish will save this content to Sanity's Content Lake (the centralized database). Once published, each document is assigned an `_id` and can be queried via the API.

Go ahead and enter some sample data for each schema. For **Project**, you might add a project titled "My Awesome App," write a short description, upload a screenshot image, provide a URL, and list some technologies. Press the **Publish** button to save each document (published documents become part of the **published** dataset accessible via queries). You can repeat this for multiple projects, blog posts, etc. Each saved item will appear in the list in Studio. (Sanity also supports draft vs published versions, but for a simple public portfolio you'll primarily work with published content.)

After publishing, you can even inspect the raw JSON data of a document in the Studio by clicking the "Inspect" option ¹¹. This is useful to see the structure of the data (for example, the image field will show a reference to an image asset document). Now that we have content in the Content Lake, let's see how to fetch it on the website.

Querying Content with GROQ

Sanity uses a query language called **GROQ (Graph-Relational Object Queries)** to retrieve content. GROQ is quite powerful and allows you to filter, project, and shape the JSON data you get from Sanity ¹². We will write GROQ queries to get our portfolio data.

Basic GROQ example: To get all Project documents with their fields, you could write:

```
*[_type == "project"]{
  title,
  description,
```

```

    projectId,
    technologies,
    image {
      asset->{
        _id,
        url
      }
    }
  }
}

```

Let's break that down: `*[_type == "project"]` means "select all documents where the `_type` is `"project"`" (i.e., all Project entries) ¹³ ¹⁴. The curly braces `{ ... }` denote a projection – which fields to retrieve and how. In the above, we list `title, description, projectId, technologies` to fetch those fields. For the `image`, we do a nested projection: `image { asset->{ _id, url } }`. This uses Sanity's dereference operator `->` to follow the reference from the image field to the actual image asset document and get its URL. In fact, Sanity's docs note you can shorthand this to `"imageUrl": image.asset->url` to directly include an `imageUrl` string in the output ¹⁵ ¹⁶. The result of such a query would be an array of objects, each representing a project with those fields. For example:

```

[
  {
    "title": "My Awesome App",
    "description": "A cool app that does amazing things...",
    "projectId": "https://myapp.com",
    "technologies": ["JavaScript", "CSS", "Sanity"],
    "image": {
      "asset": {
        "_id": "image-abc123-...-png",
        "url": "https://cdn.sanity.io/images/<projectId>/production/
abc123-400x300.png"
      }
    }
  },
  { ... next project ... }
]

```

Notice the image's `url` is an actual link to the image file on Sanity's CDN. (When you upload images, Sanity stores them and you can access them via the CDN. We projected the `url` for convenience. Alternatively, you might use Sanity's Image URL builder for dynamic transformations, but for simplicity, the direct URL works if we just need the image as-is ¹⁶.)

You can adjust GROQ queries to your needs: for example, filter or sort results (`*[_type=="project"] | order(title asc)`) to sort by title, or add conditions like `&& featured == true`). For blog posts, you might query only published posts and sort by date, etc. The GROQ syntax is quite flexible (Sanity provides a cheat sheet and docs if you need more complex queries ¹²).

Next, we'll use these queries in our website. Sanity provides two primary ways to fetch data in JavaScript: **(A)** using the official Sanity JS client library, or **(B)** using the fetch API to hit the GROQ query endpoint directly. We'll cover both.

Option A: Fetching Data with Sanity's JavaScript Client (Vanilla JS)

Sanity offers an npm package `@sanity/client` that is a convenient wrapper for querying and mutating the content. We can use this in a vanilla JS environment without a build system by including it via a CDN. The client works in the browser and Node.js alike ¹⁷ ¹⁸.

1. Include the Sanity client script: In your HTML file, before your own script that will fetch data, add:

```
<script src="https://unpkg.com/@sanity/client@latest"></script>
```

This loads the UMD build of the Sanity client. It exposes a global `SanityClient` object on `window` ¹⁹. From that, we can destructure the `createClient` factory function:

```
<script>
  const { createClient } = SanityClient;
  const client = createClient({
    projectId: "<your-project-id>", // e.g., abcdefgh (find this in
    // sanity.json or in manage.sanity.io project settings)
    dataset: "<your-dataset>", // e.g., "production"
    useCdn: true, // `true` for faster, cached responses
    apiVersion: "2025-10-31" // use today's date (YYYY-MM-DD) for API
    version 20
    // Note: useCdn:true means the client will try to fetch from Sanity's CDN
    // for faster response;
    // set false to always get latest data (bypassing CDN cache). For a static
    // site, true is fine.
    // apiVersion: It's **required** to specify a version date. Hard-code a date
    // to lock the API to that version 21.
    // (Do NOT use new Date() dynamically for apiVersion, as that defeats
    // version locking 21.)
    // If you need to query unpublished drafts or private data, you'd add a
    // token here, e.g., token: "<sanity-read-token>".
  });
</script>
```

In the above snippet, replace `projectId` and `dataset` with your actual project's ID and dataset name. You can find the project ID in the sanity configuration file (`sanity.config.ts/js` or `sanity.json`) or in the URL of your Sanity project dashboard. (If you used the CLI, it likely printed the project ID as well.) The dataset is whatever you chose (default "production"). Setting `apiVersion` is important – Sanity's API is

versioned by date; using a fixed date ensures your queries won't break if Sanity updates their API in the future ²¹. We've used today's date as an example.

2. Query with the client: Now you can use `client.fetch(groqQuery)` to retrieve data. For example, to get all projects:

```
<script>
  // ... after creating client as above ...
  const query = `*[_type == "project"]{
    title, description, projectUrl, technologies,
    "imageUrl": image.asset->url
  }`;
  client.fetch(query).then((projects) => {
    console.log("Projects data:", projects);
    // Now render the projects on the page:
    const listEl = document.getElementById("projectsList");
    listEl.innerHTML = ""; // clear any placeholder
    projects.forEach(project => {
      const item = document.createElement("div");
      item.className = "project-item";
      item.innerHTML = `
        <h3>${project.title}</h3>
        <p>${project.description}</p>
        ${project.imageUrl ? `` : ""}
        ${project.projectUrl ? `<p><a href="${project.projectUrl}"
target="_blank">View project</a></p>` : ""}
      `;
      listEl.appendChild(item);
    });
  }).catch((err) => {
    console.error("Sanity fetch error:", err);
  });
</script>
```

In this snippet, we construct a GROQ query string and call `client.fetch()`. This returns a Promise that resolves to the result (here an array of project objects). We then take that result and dynamically create some HTML to display each project: we make a `<div>` with the title, description, an image (if available), and a link. We append these to an element with id `"projectsList"` which is assumed to exist in our HTML (for example, `<div id="projectsList"></div>` in the body where we want the projects to appear). This is a simple approach for demonstration – you can structure the HTML/CSS as needed for your portfolio design.

A few things to note: - **CORS:** Since this is client-side JS calling Sanity's API, you must ensure your Sanity project allows your website's origin for requests. Sanity projects by default allow `localhost` during development, but in production you'll need to add your domain in the Sanity project settings (under **API >**

CORS Origins). You can do this in the Sanity management UI or via the CLI (`sanity cors add <your-site-origin>`). For example, in development if you test via a local server at `http://localhost:8000`, add that origin. If you skip this, you'll get CORS errors blocking the requests ²². - **Public vs Private dataset**: By default, Sanity datasets are private (requiring a token for read access). However, you can mark a dataset as public (readable by any request) in your project settings. For a portfolio, it's common to make the dataset public for convenience – then you don't need to include a token for read operations. If you keep it private, you will need to create a **read token** in Sanity (an API token with read rights) and provide it to the client: e.g., `createClient({ ..., token: "xyz" })`. Be cautious: embedding a token in front-end JS is not secure since it can be seen by anyone viewing source. If you need to use a token (to read private data or drafts), consider retrieving data in a secure environment (like at build time or through a serverless function). For purely public content, it's easiest to use a public dataset so no token is needed ²³.

- **Draft content**: The Sanity client by default fetches only published content. If you want to preview drafts, you'd have to include a token with read rights and possibly specify `{ perspective: "preview" }`, but for a live portfolio you'll likely just use published content.

Using the Sanity client SDK in the browser is straightforward and the library handles constructing URLs, parsing JSON, etc. The official docs even state that while you *can* query via HTTP directly, using a client library is recommended for convenience ²⁴. That said, we'll show the direct fetch approach too, which involves no additional library.

Option B: Fetching Data via the Sanity HTTP API (Using `fetch`)

Sanity's content lake provides a RESTful HTTP API for queries. You can call this directly with `fetch()` or any HTTP client. The endpoint format is:

```
https://<projectId>.api.sanity.io/v<YYYY-MM-DD>/data/query/<dataset>?
query=<GROQ-encoded>
```

Here, `<projectId>` and `<dataset>` are yours as before, and `vYYYY-MM-DD` is the API version date. (Using the same version date as your client would – e.g. `v2025-10-31` – is wise to ensure consistency.) Sanity also offers a CDN-powered domain `apicdn.sanity.io` which serves cached responses for queries ²⁵. If your data doesn't change often, using the CDN endpoint can speed up delivery. The format is the same except the host is `<projectId>.**apicdn**.sanity.io`.

Example: Suppose your `projectId` is `abc123xy` and dataset is `production`. A GROQ query to get project titles might look like:

```
https://abc123xy.api.sanity.io/v2025-10-31/data/query/production?
query*%5B_type%20%3D%3D%20'project'%5D%7Btitle%7D
```

(The query portion is URL-encoded. In this example, the GROQ `*[_type == 'project']{title}` was encoded to `*%5B_type%20%3D%3D%20'project'%5D%7Btitle%7D`.)

You don't need to hand-craft that encoding; you can use `encodeURIComponent` in JS. Let's write a vanilla JS snippet to fetch all projects (with some fields) using `fetch`:

```
<script>
  const projectId = "YOUR_PROJECT_ID";
  const dataset = "production";
  const apiVersion = "2025-10-31";
  const query = `*[_type == "project"]{
    title, description, projectUrl,
    "imageUrl": image.asset->url
  }`;
  const encodedQuery = encodeURIComponent(query);
  const url = `https://${projectId}.apicdn.sanity.io/v${apiVersion}/data/query/${dataset}?query=${encodedQuery}`;

  fetch(url)
    .then(response => response.json())
    .then(({result}) => {
      console.log("Projects query result:", result);
      const listEl = document.getElementById("projectsList");
      listEl.innerHTML = "";
      result.forEach(proj => {
        const item = document.createElement("div");
        item.className = "project-item";
        item.innerHTML = `
          <h3>${proj.title}</h3>
          <p>${proj.description}</p>
          ${proj.imageUrl ? `` : ""}
          ${proj.projectUrl ? `<p><a href="${proj.projectUrl}"
target="_blank">View project</a></p>` : ""}
        `;
        listEl.appendChild(item);
      });
    })
    .catch(err => {
      console.error("Error fetching Sanity data:", err);
    });
</script>
```

This is very similar to the client example, except we manually build the URL. A few notes on this approach:

- We used the `apicdn.sanity.io` host for cached results ²⁶. If you want the freshest data and don't mind slightly slower response, you could use `api.sanity.io` instead. (Setting `useCdn: false` in the client is analogous to using the non-CDN API domain.)

- The response JSON from the query endpoint comes in a `{result: ...}` envelope. We destructure to get `result`, which is the array of projects (or whatever your query returns) ²⁷ ²⁸. Always check for the structure; if you mis-write the query, sometimes you might get an error or a different shape. In the example above, we log the `result` to verify.
- **CORS and dataset access:** The same considerations apply as mentioned earlier. If your dataset is private, this request will return an error unless you include a token. You can include a token by adding an Authorization header (e.g., `fetch(url, { headers: { "Authorization": "Bearer <token>" } })`) or by adding `&token=<token>` as a query param. However, remember that exposing a token in client-side code is not secure. It's preferable to either use a public dataset or move data fetching to a secure context (like server side). Since you indicated the site is fully static and likely public, setting the dataset to public read might be simplest – then no token is needed and anyone can query it (which is fine for non-sensitive portfolio data).
- If something isn't working, check the browser console/network for errors. A 400 Bad Request could mean a query syntax error (e.g., forgetting a bracket – Sanity's API will return a 400 if the GROQ is invalid) ²⁹ ³⁰. CORS errors will appear as such in console (ensure your origin is allowed).

Using `fetch` directly is lightweight (no extra library), but you lose conveniences like auto-handling of URLs and caching that the Sanity client gives. It's a trade-off: for a small project, direct fetch is fine; the client SDK just abstracts these details. Under the hood, the client is doing similar HTTP calls for you.

Static vs Dynamic Content Fetching Considerations

You mentioned ideally having a fully static site. There are a couple of ways to interpret that: - **Fully static build (no runtime JS needed for data):** You could fetch data from Sanity at build time and bake it into your HTML. For example, you could write a Node script or use a static site generator to pull the content and generate HTML pages before deployment. This way, your pages don't even need to call Sanity's API at runtime – they already contain the content. The downside is whenever you add or change content in Sanity, you'd need to rebuild/deploy your site to see those changes live. But the site itself can be served as pure static files (great for performance and SEO). If you wanted to do this manually with vanilla JS, you could use Node.js scripts with the Sanity client. For instance, a Node script could use `@sanity/client` to fetch all projects and then write out an HTML file (or an EJS/handlebars template) for each project or section. This requires a bit of scripting and is akin to what static site generators (SSGs) do. Sanity integrates nicely with many SSGs (Next.js, Gatsby, Eleventy, Astro, etc.), but if you want to avoid frameworks, a custom script is feasible. - **Client-side fetching (dynamic data on a static page):** This is what we demonstrated above. Your site is essentially static (just HTML/JS/CSS with no server), but it uses JavaScript to fetch data from Sanity when the page loads. The benefit is you don't need a rebuild for new content – updating content in Sanity will reflect on the site whenever the site calls the API (you could even set up the pages to fetch on each load or at intervals). The trade-off is a slightly added complexity: the page load has an extra step (fetching from Sanity), and you need to handle the asynchronous rendering. Also, if the dataset is private, you'd have to expose a token or implement a proxy, which is not ideal. For a public portfolio, making the dataset public sidesteps that, as discussed.

Both approaches ultimately use Sanity as the content source; you can even mix them (e.g., load some content at build time and some dynamically). For simplicity, many people building portfolios opt for client-side fetches because it avoids writing a build pipeline. The site remains static in deployment (just files), and

Sanity's CDN provides the data on the fly. Given that your site is likely small, the performance impact is minimal (especially with `useCdn`, the data is served fast from a global CDN ²⁵).

If you ever decide to use a framework or SSG (like Next.js, which can do static generation with Sanity, or Eleventy), Sanity has guides and starters. But since the goal here is **vanilla JS minimalism**, the approach we've outlined (fetching via the API or client in the browser) keeps things lightweight.

Final Steps and Summary

To recap the workflow: 1. **Set up Sanity Studio and project:** Use `npm create sanity@latest` to initialize, choose the clean starter ⁴, and define your schemas for portfolio content. Run `npm run dev` to launch the Studio locally ³¹. 2. **Add content in Studio:** In the Studio UI, create documents for your projects, blog posts, etc., and publish them. Ensure your dataset is public (or configure a read token) for easy access. 3. **Fetch content on your site:** In your static site's HTML/JS, use either the Sanity JS client or `fetch` to query the Content Lake. We provided code examples for both methods. The GROQ queries will retrieve your content as JSON. 4. **Render the content:** Take the returned JSON (array of projects, etc.) and use DOM manipulation (or templating) to insert it into your page structure. We used simple `innerHTML` and element creation to demonstrate. 5. **Deployment:** Since your site is static (no server needed aside from serving static files), you can host it on any static hosting (GitHub Pages, Netlify, Vercel, etc.). Just be sure to add the production site's URL to Sanity CORS allowed origins. When you update content in Sanity, those changes will appear on the live site whenever the pages fetch new data (e.g., on reload).

With this setup, you have a content-managed portfolio powered by Sanity, using only vanilla JavaScript on the front-end. You avoid heavy frameworks, keeping the implementation minimal, yet you gain the ability to easily update your portfolio through Sanity's friendly interface. The detailed steps and code above should give Codex (or yourself) enough context to start coding the integration. Happy coding, and enjoy your new Sanity-powered portfolio!

Sources: The information and code examples above were based on Sanity's official documentation and community guides, adapted for a vanilla JS context. Key references include Sanity's docs on installation ³², schema design and content queries ⁷ ¹⁵, and the Sanity JS client usage ³³, as well as community Q&A about using Sanity with plain JS ³⁴ ²⁹. These sources provide further details on each aspect if you need to dive deeper.

1 12 **Store and query structured content | Sanity Docs**

<https://www.sanity.io/docs/content-lake>

2 31 32 **Installation | Sanity Docs**

<https://www.sanity.io/docs/studio/installation>

3 4 5 **Clarification on different ways to set up a Sanity project**

<https://www.sanity.io/answers/clarification-on-different-ways-to-set-up-a-sanity-project>

6 7 8 9 10 11 **How to Build a Portfolio Site with Sanity and Next.js**

<https://www.freecodecamp.org/news/how-to-build-a-portfolio-site-with-sanity-and-nextjs/>

13 14 24 25 26 **Query | Sanity Docs**

<https://www.sanity.io/docs/http-reference/query>

15 16 **Presenting Images | Sanity Docs**

<https://www.sanity.io/docs/apis-and-sdks/presenting-images>

17 18 19 20 21 33 **GitHub - sanity-io/client: JavaScript client for retrieving, creating and patching data from Sanity.io**

<https://github.com/sanity-io/client>

22 27 28 29 30 34 **Discussion about CORS errors when making HTTP/HTTPS requests in a Sanity.io...**

<https://www.sanity.io/answers/discussion-about-cors-errors-when-making-http-https-requests-in-a-sanity-io-app->

23 **Understanding private datasets and read tokens in Sanity.io**

<https://www.sanity.io/answers/understanding-private-datasets-and-read-tokens-in-sanity-io>